a JavaScript UI that relies heavily on JSON, it can make sense to provide sample JSON output as part of the BDD scenarios. In this environment, the target audience of the living documentation will be both the developers who are implementing the UI and the business, which will need to understand the business logic. If the JSON is simple and clean enough to make sense to both parties, there's no reason not to include the JSON output directly in the scenario:

```
Scenario: Should return flight details in JSON form
  Given I need to know the details of flight number FH-102
  When I request the details about this flight in JSON format
  Then I should receive:
  """
  {
    "flightNumber":"FH-102",
    "departure":"SYD",
    "destination":"MEL",
    "time":"06:15"

  }
  """
```

A known flight

The expected JSON result

You could automate these steps in a similar way to the previous example:

```
String receivedJsonData;

@When("^I request the details about this flight in JSON format$")
public void request_details_in_json_format() {
    receivedJsonData = client.findByFlightNumberInJson(flightNumber);
}

@Then("^I should receive:$")
public void should_receive_json_data(String expectedJsonData)
                                                    throws JSONException {
    JSONAssert.assertEquals(expectedJsonData,
                            receivedJsonData,
                            JSONCompareMode.LENIENT);
}
```

Retrieve the search results as a JSON document.

Compare the retrieved JSON with the expected data using the JSONassert library.

Here you use the JSONassert library (https://github.com/skyscreamer/JSONassert) to compare the JSON data you receive from the web service with that given in the feature file. And once again, you delegate the actual code that invokes the web service to the `FlightStatusClient` class you saw earlier:

```
public String findByFlightNumberInJson(String flightNumber) {
    Client client = ClientBuilder.newClient();
    WebTarget webTarget = client.target(BASE_URL).path(flightNumber);
    return webTarget.request().buildGet().invoke(String.class);
}
```

Return the result in raw JSON format.

This is a simple example, but JSON is relatively readable, and more complicated scenarios are quite possible without compromising readability. The following scenario provides test data to be injected into the flight schedule database: